

✓ Problem Statement : Implementation of BERT for Text

Dataset Used : <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

```
# Importing Necessary Libraries :
import numpy as np
import pandas as pd
import csv

import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import AdamW
from torch.utils.data import Dataset, DataLoader

import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from transformers import BertTokenizerFast, TFBertForSequenceClassification
from transformers import BertForSequenceClassification, Trainer, TrainingArguments
from transformers import BertTokenizer, TFBertModel
from transformers import BertTokenizer, BertForSequenceClassification
from tqdm import tqdm

import warnings
warnings.filterwarnings('ignore')

# Use GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

```
device(type='cuda')
```

```
# 1. Load IMDb CSV
```

```
df = pd.read_csv("/content/drive/MyDrive/IMDB Dataset.csv")
df['label'] = df['sentiment'].map({'positive': 1, 'negative': 0})
```

```
# 2. Split into train/test
```

```
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['review'].values, df['label'].values, test_size=0.2, random_state=42
)
```

```
# 3. Tokenization
```

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.91kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 1.38MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 917kB/s]
config.json: 100% 570/570 [00:00<00:00, 23.6kB/s]
```

```
class IMDbDataset(Dataset):
```

```
    def __init__(self, texts, labels, tokenizer, max_len=512):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len
```

```
    def __len__(self):
        return len(self.texts)
```

```
    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
```

```
        max_length=self.max_len,
        return_tensors='pt'
    )
    item = {key: val.squeeze(0) for key, val in encoding.items()}
    item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
    return item

train_dataset = IMDBDataset(train_texts, train_labels, tokenizer)
test_dataset = IMDBDataset(test_texts, test_labels, tokenizer)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)

# 4. Load BERT model
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)
model.to(device)
```

 Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance, you should install the 'hf_xet' package.
 WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download.
 model.safetensors: 100% 440M/440M [00:01<00:00, 336MB/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSdpaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
        )
      )
      (intermediate): BertIntermediate(
        (dense): Linear(in_features=768, out_features=3072, bias=True)
        (intermediate_act_fn): GELUActivation()
      )
      (output): BertOutput(
        (dense): Linear(in_features=3072, out_features=768, bias=True)
        (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (dropout): Dropout(p=0.1, inplace=False)
      )
    )
  )
)
(nooler): BertPooler(
```

```

        (dense): Linear(in_features=768, out_features=768, bias=True)
        (activation): Tanh()
    )
)
(dropout): Dropout(p=0.1, inplace=False)
(classifier): Linear(in_features=768, out_features=2, bias=True)
)

```

5. Optimizer

```
optimizer = AdamW(model.parameters(), lr=2e-5)
```

6. Training Loop

```
EPOCHS = 2
```

```

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    for batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}"):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        total_loss += loss.item()
    print(f"Epoch {epoch+1} Loss: {total_loss/len(train_loader):.4f}")

```

```

➡ Epoch 1/2: 100%|██████████| 2500/2500 [1:03:37<00:00, 1.53s/it]
Epoch 1 Loss: 0.2105
Epoch 2/2: 100%|██████████| 2500/2500 [1:03:37<00:00, 1.53s/it]Epoch 2 Loss: 0.1096

```

7. Evaluation

```

model.eval()
all_preds, all_labels = [], []

```

```

with torch.no_grad():
    for batch in test_loader:
        batch = {k: v.to(device) for k, v in batch.items()}

```

```
outputs = model(**batch)
preds = torch.argmax(outputs.logits, dim=1)
all_preds.extend(preds.cpu().numpy())
all_labels.extend(batch['labels'].cpu().numpy())

acc = accuracy_score(all_labels, all_preds)
print(f"Test Accuracy: {acc:.4f}")
```

 Test Accuracy: 0.9440